

GetSafe Analytics Case Study 2

Flow Chart



Purpose

The purpose of this project is to design and implement a reusable analytics engineering solution that produces a reporting data mart for customer contract KPIs. The mart enables the business to track the number of active customers per day and product group, the premium collected from active customers over any date range, and the accumulated acquired premium from inception to a given date. The solution is implemented in dbt and BigQuery.

Data Overview

A screenshot is provided which contains raw products_customers data (contracts).

File name in dbt seed folder - sample_products_customers.csv

`getsafe-bigquery-prod.getsafe_data_raw.sample_products_customers` - This table represents the contractual relationship between a user and Getsafe for a given product group and defines the contracts data used to create a closing report for the BI team or stakeholders.

Key Dimensions & Metrics

Core Dimensions

acquisition_date, started_at, churned_at, user_id, product_group, gen_date, hash_key

Engagement Metrics

active_customer, active_monthly_premium, active_daily_premium

Process

The complete process is implemented in DBT and Bigquery, following a layered architecture.

1. The CSV file is uploaded into the seed folder in dbt where we run a dbt seed command to upload the file in BigQuery as raw table - `sample_products_customers` under `getsafe_raw` dataset. This raw table serves as the immutable source layer and is not directly consumed for analysis.

2. The raw table is transformed into curated staging models using dbt. This curated table serve as the clean and standardized foundation layer. The following transformations were implemented:
 - a. Converted column names to snake_case.
 - b. Updated columns to appropriate data types like date, float, string, etc.
 - c. Created a unique hex key named hash key ensuring the uniqueness of each row.
3. To optimize transformation logic and maintain modularity, intermediate models were created using ephemeral materialization in dbt. This helped the pipeline to avoid unnecessary physical table creation, improve query performance and readability, reduced storage overhead. These ephemeral models are compiled into downstream sql at runtime and are not materialised in Bigquery. Ephemeral model names are `product_customer_source_enriched` , `create_date_spine` .
4. A reusable date spine is created to support flexible date ranges at daily granularity. This generates a continuous series of dates between a configurable min and max date. Code snippet is below:

```
1 with bounds as (  
2     select  
3         min(started_at) as min_date,  
4         max(coalesce(churned_at, current_date())) as max_date  
5     from {{ source('getsafe_curated', 'getsafe_contracts_curated') }}  
6 ),  
7  
8 dates as (  
9     select  
10         gen_date  
11     from bounds,  
12         unnest(generate_date_array(min_date, date_add(max_date, interval 1  
13 day),  
14         interval 1 day)) as gen_date  
15 )
```

5. This intermediate model expands each contract into one row per active day. In this I am joining the contracts data to the date spine where the gen_date falls between `started_at` and ``churned_at`` . If the churned date is null then it is considered as an active account till date. Code snippet below:

```
1 with contracts as (  
2     select *  
3     from {{ source('getsafe_curated', 'getsafe_contracts_curated') }}  
4 ),  
5  
6 dates as (  
7     select gen_date  
8     from {{ ref('create_date_spine') }}  
9     {% if is_incremental() %}  
10         where gen_date >= date_sub(current_date(), interval 90 day)  
11     {% endif %}  
12 ),
```

```

13
14 data_joined as (
15     select
16         d.gen_date,
17         c.hash_key,
18         c.user_id,
19         c.product_group,
20         c.acquisition_date,
21         c.started_at,
22         c.churned_at,
23         c.monthly_premium
24     from contracts c
25     inner join dates d
26         on d.gen_date >= c.started_at
27         and (c.churned_at is null or d.gen_date <= c.churned_at)
28 )

```

This produces one row per user per product_group and get_date while the contract is active.

- The final contracts reporting model computes the metrics in a way that the BI tool only needs sum and count logic. Columns derived from the model are active_customers = 1 for each active row, active_daily_premium, and active_monthly_premium logic as below

```

1 1 as active_customers,
2 safe_divide(monthly_premium, extract(day from last_day(gen_date)))
3 ) as active_daily_premium,
4 case
5     when gen_date = started_at then monthly_premium
6     when extract(day from gen_date) = 1 then monthly_premium
7     else 0
8 end as active_monthly_premium

```

- The reporting table supports the BI team to calculate Active customers per day per product group in a given active period, premium collected from active customers in a given range, and accumulated acquired premium until a given date.
- The BI tool then connects with the reporting table to get the desired values using only sum and count aggregation. No complex SQL or Window function required. You can find the exact code in my repository mentioned under the folder name 'getsafe_intermediate'.

Tables Created

Layer	Dataset	Table
Raw layer	getsafe- bigquery- prod.getsafe_da ta_raw	sample_products_cus tomers
Curated layer	getsafe- bigquery-	getsafe_contracts_c urated

	prod.getsafe_cu rated	
Reporting Layer	getsafe- bigquery- prod.getsafe_re porting	getsafe_contracts_r eporting

Reporting Results

1. Number of active customers per day and product group in a given date range.

```
1 SELECT product_group, gen_date, count(distinct user_id) as active_user FROM
2 `getsafe-bigquery-prod.getsafe_reporting.getsafe_contracts_reporting`
3 where started_at >= '2024-01-01' and churned_at is null group by all
```

product_group	gen_date	active_user
car	2026-06-15	1
contents	2026-06-15	1
dog	2026-06-15	2
legal	2026-06-15	1
liability	2026-06-15	2
car	2026-06-14	1
contents	2026-06-14	1

2. Premium collected from active customers in a given date range.

```
1 SELECT user_id, round(sum(active_monthly_premium),2) as premium FROM
2 `getsafe-bigquery-prod.getsafe_reporting.getsafe_contracts_reporting`
3 where started_at >= '2024-01-01' and churned_at is null group by all
```

user_id	premium
1-088f	588.48
9-918f	581.7
0-3594	559.13
1-18c6	553.25
5-169e	342.24
d-604b	325.6
9-8903	5.7

3. Accumulated acquired premium from the beginning of time until a given date.

```
1 SELECT round(sum(active_monthly_premium),2) as premium FROM
2 `getsafe-bigquery-prod.getsafe_reporting.getsafe_contracts_reporting`
3 where gen_date between '2024-09-01' and '2026-01-01' group by all
```

premium
1 1938.7

Challenges

1. Partitioning of data by a date column can help reduce table scan volume. As I am using a Bigquery non billable account the partitions are automatically reduced to last 60 days which was creating a data upload discrepancies as the dates provided in the raw data are from last year or are older than the last 60 days. (But I have mentioned the partition_by clause in the code before committing so you will be able to replicate at your end.)
2. As the partitioning is a challenge, testing the incremental logic was difficult to conclude.

Further Data Improvements

1. Create a DAG or a scheduler for daily refresh.
2. Create incremental dbt model on column 'started_at' to reduce the processing cost.
3. Store large csv files in the google cloud bucket to maintain the raw data.

Proposed Approach 2

1. Adding 1 column - 'create_timestamp' or 'load_timestamp' which define the time when the new data was being uploaded in the raw table. This will help us maintain a historical data when the churn date changes in the past and the company has to refund the amount.
2. Next following the same steps for the curated layer.
3. For the reporting layer creating incremental pipeline on 'started_at' and 'hash_key' to keep the logic more cost effective.

Repository Details

You can find the complete code structure and logic here -

[GitHub - PreetikaJ/getsafe_analytics_dbt_mart: Analytics Engineering case study for Getsafe, implementing scalable dbt models for premium reconciliation and KPI data marts. Covers financial data modeling, discrepancy analysis, and customer lifecycle metrics optimized for BI consumption.](#)